Nemo: Efficient Caching with Index Pushdown for Disaggregated OLTP Databases

https://arxiv.org/pdf/2610.33306

1. Problem

- 핵심 문제: 컴퓨팅과 스토리지가 분리된(Disaggregated) OLTP 데이터베이스에서 네트워크 왕복 지연과 비효율적인 캐싱으로 인해 쿼리 성능이 저하됨.
- 왜 중요한가? 기존 페이지 단위 캐싱은 불필요한 데이터 전송을 유발하고, 인덱스 탐색마다 발생하는 원격 통신이 병목이 됨.
- 추가 난제: 존재하지 않는 키 검색(Negative Search)의 지연과 Phantom 현상을 방지하는 동시성 제어 문제까지 해결해야 함.

2. Method

- 핵심 아이디어: 인덱스는 스토리지 측으로 내려보내고, 컴퓨팅 측에서는 필요한 레코드만 효율적으로 캐싱하는 Nemo 구조를 제안.
- 기존 방법과의 차이: 기존 페이지 기반 B+Tree 및 버퍼 풀 대신, 레코드 단위 캐싱과 인덱스 푸시다운을 결합함.
- 핵심 메커니즘 3가지
- Tree-structured Record Buffer: 레코드 캐시와 버퍼를 통합해 Point Query와 Range Query를 모두 가속.
- Negative Search Caching: 존재하지 않는 키의 검색 결과도 캐싱해 반복적인 원격 조회를 줄임.
- Storage-agnostic Phantom Protection: 스토리지 내부 데이터 배치 구조를 알지 못해도 Phantom 방지를 지원하며, 키 도메인 크기와 데이터 분포 변화에 대응.

3. Experiments & Results

평가 항목	주요 결과
Point Query	페이지 기반 B+Tree 대비 최대 11.42배 향상
Point Query	기존 2-Tree 대비 최대 2.97배 향상
Negative Search	해시 기반 레코드 캐시 대비 최대 4.76배 향상
Phantom Protection	PCL 대비 최대 1.55배 향상

- 무엇이 개선됐는가?
  - 레코드 단위 캐싱으로 네트워크 전송 및 캐시 효율 개선.
  - Negative Search 결과 캐싱으로 불필요한 원격 조회 감소.
  - 키 공간이 크고 희소해지는 상황에서도 Phantom Protection 성능을 안정적으로 유지.
- 실험에서 주목할 점: 키 도메인이 커지고 희소해질 때 기존 PCL의 성능은 크게 저하되지만, Nemo는 상대적으로 안정적인 성능을 보임.

4. Insight

- 한계점 및 아쉬운 점
  - 실제 워크로드 구성, 하드웨어 환경, 메모리 사용량, 쓰기 부하에서의 성능 및 전체 실험 조건을 충분히 평가하기 어려움.
  - 최대 성능 향상 수치만으로 평균적인 성능이나 실제 서비스에서의 개선 폭을 판단하기는 어려움.
- 내가 가져갈 포인트
  - 캐싱 최적화는 단순히 캐시 적중률을 높이는 것만으로 충분하지 않음. 네트워크 왕복 횟수, 데이터 전송 단위, 캐시 미스 처리 비용을 함께 최적화해야 함.
  - 캐시에 데이터뿐 아니라 검색 결과의 부재까지 저장하면 반복적인 원격 조회를 줄일 수 있음.
  - 분산 시스템에서는 성능뿐 아니라 데이터 레이아웃에 대한 의존성을 줄이면서 동시성 정확성을 보장하는 설계도 중요함.